

Gabriela Mora, V-31.541.893
Lucía Martínez, V-31.906.577
Maivet Pereira, V-30.814.708

Simulador de Caché de Correspondencia Directa y Asociativa por Vías

Arquitectura del Computador

1 de Septiembre de 2026

El Problema

Descripción del problema

La memoria caché es un pequeño espacio de almacenamiento ultrarrápido que el procesador utiliza para evitar ir constantemente a la memoria principal, que es mucho más lenta. El problema es que su tamaño es limitado y, si no se gestiona bien, el procesador pasa mucho tiempo esperando datos. Este proyecto simula su comportamiento para entender cómo optimizar su uso.

Objetivos del proyecto

Desarrollar un simulador en C++ que modele el funcionamiento de una caché, permitiendo probar diferentes configuraciones (mapeo directo y asociativo por vías) y analizar su rendimiento en términos de aciertos y fallos.

Alcance

Nuestro simulador trabaja con dos tipos de organización: la caché de mapeo directo, donde cada bloque de memoria solo puede ir a un lugar específico, y la

caché asociativa por vías, que ofrece más flexibilidad al permitir que un bloque se ubique en varias líneas dentro de un conjunto.

Técnicas Computacionales para su Solución

Estructuras de Datos en C++

La caché se modela utilizando una matriz de estructuras `lineaCache` con `std::vector`:

- `const int tamanoBloque = 4;` Constante que define el tamaño del bloque en palabras. Cada bloque almacena 4 palabras.
- `struct lineaCache`: Representa un bloque de caché, conteniendo los campos `datos` (array de tamaño `tamanoBloque`), `tag`, `ultimoUso` y `valido`.

```
const int tamanoBloque = 4;

struct lineaCache
{
    int datos[tamanoBloque]; //array para almacenar un bloque completo
    int tag;
    int ultimoUso; //contador de reloj para LRU
    bool valido;

    lineaCache(): tag(-1), ultimoUso(0), valido(false)
    {
        for(int i = 0; i < tamanoBloque; i++){
            datos[i] = 0;
        }
    }
};
```

- `vector<vector<lineaCache>> conjuntos;` Se utiliza una matriz de vector donde cada fila (conjunto) contiene un vector de líneas (vías). Esto permite un acceso directo por índice en tiempo constante $O(1)$.
- `vector<int> direcciones;` Un arreglo dinámico para almacenar las direcciones ingresadas por el usuario.

Algoritmos de Simulación

Cálculo de Índice, Tag y Offset: Se utilizan operaciones aritméticas (`%` y `/`) para calcular el conjunto, la etiqueta y el desplazamiento dentro del bloque a partir de la dirección, considerando el tamaño de bloque.

```
int indice = (direccion / tamanoBloque) % numConjuntos;
int tag = direccion / (tamanoBloque * numConjuntos);
int offset = direccion % tamanoBloque; //desplazamiento dentro del bloque
```

LRU (Least Recently Used)

El LRU es el algoritmo de reemplazo que usamos para decidir qué línea eliminar cuando la caché está llena. Su funcionamiento es sencillo: cada línea de caché tiene un contador llamado ultimoUso que se actualiza cada vez que se accede a ella. Cuando ocurre un fallo y no hay líneas libres, simplemente reemplazamos la línea con el valor más bajo en ultimoUso, es decir, la que lleva más tiempo sin ser utilizada.

```
//si no hay lineas vacias, se aplica la politica de reemplazo LRU
int lru = 0;

//recorre todas las vias para encontrar la de menor ultimoUso (la mas antigua)
for (int i = 1; i < vias; i++)
{
    if (conjuntos[indice][i].ultimoUso < conjuntos[indice][lru].ultimoUso)
    {
        lru = i; //actualiza el indice de la linea LRU
    }
}

//reemplaza la linea LRU con el nuevo bloque
conjuntos[indice][lru].tag = tag;
for(int j = 0; j < tamanoBloque; j++)
    conjuntos[indice][lru].datos[j] = direccion - offset + j;
conjuntos[indice][lru].valido = true;
conjuntos[indice][lru].ultimoUso = reloj;
```

Prefetching (Anticipo de Datos)

El prefetching es una técnica que usamos para anticipar las necesidades del procesador. Cuando el prefetch está activado, no solo cargamos la dirección solicitada, sino también los bloques siguientes ($\text{direccion} + \text{tamanoBloque}$, $+2*\text{tamanoBloque}$, etc.). Esto aprovecha la localidad espacial, ya que los programas suelen acceder a datos cercanos en memoria. El resultado es una reducción significativa de los fallos, especialmente en patrones de acceso secuencial.

```
void precargar(int direccion, int cantidad)
{
    for (int p = 1; p <= cantidad; p++)
    {
        //calcula la direccion del siguiente bloque (multiplicando por tamanoBloque)
    }
}
```

```

int dirBloque = direccion + (p * tamanoBloque);
int indice = (dirBloque / tamanoBloque) % numConjuntos;
int tag = dirBloque / (tamanoBloque * numConjuntos);

bool encontrado = false;

//verifica si el bloque ya esta en la cache
for (int i = 0; i < vias; i++)
{
    if (conjuntos[indice][i].valido && conjuntos[indice][i].tag == tag)
    {
        encontrado = true;
        //NO actualizamos ultimoUso para no distorsionar LRU
        break;
    }
}

//si no esta, lo carga
if (!encontrado)
{
    bool asignado = false;

    //primero busca una via vacia
    for (int i = 0; i < vias && !asignado; i++)
    {
        if (!conjuntos[indice][i].valido)
        {
            conjuntos[indice][i].tag = tag;
            for(int j = 0; j < tamanoBloque; j++)
                conjuntos[indice][i].datos[j] = dirBloque + j;
            conjuntos[indice][i].valido = true;
            conjuntos[indice][i].ultimoUso = reloj;
            asignado = true;
        }
    }

    //si no hay vias vacias, usa LRU
    if (!asignado)
    {
        int lru = 0;
        for (int i = 1; i < vias; i++)
        {
            if (conjuntos[indice][i].ultimoUso < conjuntos[indice][lru].ultimoUso)
            {
                lru = i;
            }
        }

        conjuntos[indice][lru].tag = tag;
        for(int j = 0; j < tamanoBloque; j++)
            conjuntos[indice][lru].datos[j] = dirBloque + j;
        conjuntos[indice][lru].valido = true;
        conjuntos[indice][lru].ultimoUso = reloj;
    }
}
}

```

Organización de la Caché y Experimentación

El simulador mantiene un total de 16 líneas, permitiendo probar diferentes organizaciones:

- **Mapeo Directo:** Cuando se usa $vias = 1$, cada dirección solo puede residir en un conjunto específico.
- **Asociativa por Vías:** Al aumentar el número de vías (2, 4, 8, 16), los conjuntos se reducen, pero hay más flexibilidad para ubicar los datos, lo que reduce los fallos de conflicto.

Interfaz de Usuario

Se implementó un menú interactivo que permite:

- Cargar direcciones desde un archivo de texto (`entradaCache.txt`).
- Ver las estadísticas de la caché (aciertos, fallos, tasa).
- Comparar el rendimiento de diferentes esquemas (mapeo directo vs. asociativa) con o sin prefetch.

Análisis y Discusión de Resultados

Experimentos Realizados

Probamos diferentes tipos de secuencias de direcciones para evaluar el comportamiento de la caché bajo distintas condiciones de localidad espacial y temporal. Cada secuencia consta de 30 accesos y se probó con cachés de 1, 2, 4 y 8 vías (manteniendo un total de 16 bloques), con y sin prefetch de 1 bloque.

Resultados Obtenidos

Table 1: Tasas de acierto sin prefetch

Tipo de Secuencia	Directa (1 vía)	2 vías	4 vías	8 vías
Con localidad temporal	66.67%	66.67%	66.67%	66.67%
Con localidad espacial	0%	0%	0%	0%
Sin localidad (aleatoria)	0%	0%	0%	0%

Table 2: Tasas de acierto con prefetch (1 bloque)

Tipo de Secuencia	Directa (1 vía)	2 vías	4 vías	8 vías
Con localidad temporal	96.67%	96.67%	96.67%	96.67%
Con localidad espacial	50%	50%	50%	50%
Sin localidad (aleatoria)	0%	0%	0%	0%

Análisis de Resultados

Los experimentos realizados demuestran que:

- El prefetch mejora significativamente la tasa de aciertos cuando hay localidad espacial (direcciones cercanas), como se ve en las secuencias con localidad espacial y temporal.
- El prefetch no aporta beneficio cuando las direcciones están muy espaciadas y no hay localidad espacial (secuencia aleatoria).
- La localidad temporal (repetición de direcciones) es el factor más importante para una alta tasa de aciertos, alcanzando un 66.67% sin prefetch y 96.67% con prefetch.
- El tamaño de bloque de 4 palabras permite aprovechar la localidad espacial de manera efectiva, permitiendo que el prefetch anticipe correctamente los bloques siguientes.

La caché de 8 vías dió la mejor tasa en todos los casos por su mayor flexibilidad para ubicar datos. El prefetch mejoró los resultados en secuencias con localidad espacial y temporal (aprovechando la localidad espacial), pero no tuvo efecto en la aleatoria. Esto confirma que el prefetch es útil solo cuando los accesos son predecibles.

Conclusiones

El desarrollo de este simulador nos permitió comprender en profundidad el funcionamiento de la memoria caché y los factores que influyen en su rendimiento.

Logros Alcanzados

- Implementamos un simulador funcional que modela correctamente una caché de mapeo directo y asociativa por vías, permitiendo probar diferentes configuraciones y analizar su rendimiento.

- Logramos integrar el algoritmo de reemplazo LRU, que demostró ser efectivo para gestionar el espacio limitado de la caché.
- Implementamos la técnica de prefetch, que resultó beneficiosa en patrones de acceso con localidad espacial, cumpliendo con el objetivo de anticipar la carga de datos.
- Desarrollamos una interfaz interactiva que facilita la experimentación con diferentes configuraciones y secuencias de direcciones.

Dificultades Encontradas

- La implementación correcta del algoritmo LRU requirió especial atención para garantizar que las decisiones de reemplazo fueran óptimas.
- Manejar el tamaño de bloque en todos los cálculos (índice, tag, offset, prefetch) fue un desafío que nos obligó a ser muy cuidadosas con las operaciones aritméticas.
- Coordinar el trabajo en equipo y mantener la consistencia del código fue una dificultad que nos ayudó a mejorar nuestras habilidades de colaboración y comunicación.

Este proyecto no solo nos permitió cumplir con los objetivos académicos, sino que nos brindó una comprensión práctica de conceptos fundamentales de arquitectura de computadores. Aprendimos que el diseño de una caché implica un delicado equilibrio entre capacidad, velocidad y complejidad, y que las técnicas como el prefetch y el LRU son herramientas poderosas cuando se utilizan correctamente.